

Lecture 4: Functions of Random Variables, Linearity, Variance

Jamie Haddock

Table of contents

- 1 LOTUS and linearity::: {.callout-warning icon=false}## Theorem: Law of the Unconscious Statistician (LOTUS)

$$E[g(X)] = \sum_x g(x) p_X(x)$$

– no need to first find the distribution of $g(X)$.::: . . ::: {.callout-warning icon=false}## Theorem: Linearity of ExpectationFor *any* random variables $X_1, \dots, X_n$ (independent or not!) and constants a_i ,

$$E\left[\sum_i a_i X_i\right] = \sum_i a_i E[X_i].$$

:::—::: {.callout-note icon=false}## Definition: Variance

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2, \quad \text{Var}(aX + b) = a^2 \text{Var}(X).$$

:::## Linearity in action: fixed points of a random permutationShuffle n cards; let X = number of cards in their original position. X is a sum of *dependent* indicators, but linearity doesn't care! 2

—title: “Lecture 4: Functions of Random Variables, Linearity, Variance”author: “Jamie Haddock”format: revealjs: output-file: Lecture4_slides slide-number: true preview-links: auto logo: figs/hmc.png css: input/slides.css incremental: true smaller: true code-fold: true embed-resources: true html: code-fold: true embed-resources: true pdf: documentclass: article toc: true number-sections: true geometry: - top=1in - left=1in - bottom=1in - right=1informat-links: falsejupyter: python3filters: - input/remove-pause.luaexecute: echo: true eval: true—

- 1 LOTUS and linearity::: {.callout-warning icon=false}## Theorem: Law of the Unconscious Statistician (LOTUS)

$$E[g(X)] = \sum_x g(x) p_X(x)$$

– no need to first find the distribution of $g(X)$.::: . . ::: {.callout-warning icon=false}## Theorem: Linearity of ExpectationFor *any* random variables $X_1, \dots, X_n$ (independent or not!) and constants a_i ,

$$E\left[\sum_i a_i X_i\right] = \sum_i a_i E[X_i].$$

---: {callout-note icon=false}## Definition: Variance

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2, \quad \text{Var}(aX + b) = a^2 \text{Var}(X).$$

Linearity in action: fixed points of a random permutation-
Shuffle n cards; let X = number of cards in their original position. X is a sum of *dependent* indicators, but linearity doesn't care!

```
n = 52n_trials = 20000fixed_points = [np.sum(rng.permutation(n) == np.arange(n)) for _ in range(n_trials)]
```

—# Case Study: Randomized Trace EstimationFor $A \in \mathbb{R}^{n \times n}$, let z be a **Rademacher** vector (entries iid ± 1). Hutchinson's estimator is $T = z^\top A z$.: {callout-warning icon=false}## Theorem: Unbiasedness of the trace estimator $E[T] = \text{tr}(A)$:

Proof:

Using $E[z_i z_j] = 0$ for $i \neq j$ and $E[z_i^2] = 1$, linearity gives

$$E[T] = E\left[\sum_{i,j} z_i A_{ij} z_j\right] = \sum_{i,j} A_{ij} E[z_i z_j] = \sum_i A_{ii} = \text{tr}(A).$$

```
n = 50A = rng.normal(size=(n, n))true_trace = np.trace(A)def hutchinson_estimate():    z = rng.choice([
```

Next week we'll average several samples together, once we've discussed covariance.## Reading & HomeworkReading: §2.4-2.5 (*Bertsekas & Tsitsiklis, Introduction to Probability*).